

Unexpected Cases Observed in Smoke Testing

This document records three behaviors observed in `gemini-judge-smoke-3-03`. They are useful warnings for future benchmark and experiment analysis. They should not be interpreted as general model results: the run contained only five C programs and was intended to test the pipeline.

Summary

Case	What behaved unexpectedly	How the pipeline handled it
p02203	Validator repair guidance misunderstood one <code>scanf</code> call and damaged two initially accepted translations.	The initial and final Judge results exposed the regression, but the repair loop still repeated it until the repair limit.
p00639 Judge	The accepted C program was rejected because outputs requiring numeric tolerance were compared textually.	The C baseline was marked invalid, so final Rust evaluation was safely recorded as SKIPPED.
p00639 visible tests	Most generated inputs omitted the terminating sentinel and timed out; the frozen suite contained only the termination case.	Deterministic filtering rejected the timeouts. The old whole-suite regeneration then lost an otherwise useful survivor. The current flow preserves passing slots, regenerates failed slots, and reviews the retained batch as a whole.

1. Incorrect `scanf` Interpretation Caused Regressive Repairs

What happened

In `p02203`, the C source contains:

```
scanf("%llu", &n, &m);
```

The format string contains one conversion specifier, so the call consumes one input value and assigns only `n`. The extra destination argument does not cause a second value to be read.

The Validator incorrectly diagnosed the call as consuming both `n` and `m`. Its repair guidance instructed the Translator to consume an additional token in Rust. That shifted the remaining input and damaged correct translations.

Two translations that initially passed all 20 Judge cases regressed:

Prompt	Initial Judge	Final Judge after repairs
direct-v1	ACCEPTED, 20/20	RUNTIME_ERROR, 0/20
trace-performance-v1	ACCEPTED, 20/20	WRONG_ANSWER, 8/20

How the program handled it

- The visible-test difference triggered Validator analysis and Translator repair, as designed.

- The same incorrect interpretation continued across five repair attempts.
- Reaching the repair limit no longer prevented the final Judge stage.
- Because initial code and initial Judge evidence were preserved separately, the reports clearly showed that repair introduced the regression.
- Judge evidence remained observational and did not start another repair.

Current consideration

This is a source-level irregularity in one accepted submission, not a rule the experiment should encode around. It remains documented as a limitation of model-based repair guidance. The run also demonstrates why initial and final Judge results must remain separate in analysis.

2. Numeric-Tolerance Output Was Rejected Textually

What happened

p00639 , *Accelerated Railgun*, allows any number of output digits as long as the numeric error is below $1.0e-6$. The selected accepted C program prints eight decimal places, while the downloaded AOJ reference output contains twelve.

The local Judge used textual comparison after whitespace normalization. It therefore reported `WRONG_ANSWER` even though:

- all 79 output tokens corresponded;
- there were no non-numeric differences;
- the maximum absolute numeric difference was approximately $4.95e-9$, well below the permitted error.

The problem mapping itself was correct: the CodeNet and AOJ descriptions had an exact matching hash.

How the program handled it

- The accepted C program was run before trusting the Judge suite.
- Its local `WRONG_ANSWER` marked the baseline as invalid.
- The pipeline still reached the final Evaluator stage, but Rust execution was not treated as a valid comparison.
- Every combination for this problem received final Judge status `SKIPPED` and failure category `invalid_baseline` .
- These results do not enter the accepted or rejected translation totals.

Current consideration

`SKIPPED` correctly protected the experiment from an unreliable local oracle, but the underlying Judge limitation remains. Problems that require floating-point tolerance or another special checker need a supported comparator or an explicit unsupported-checker classification before the definitive experiment.

3. Missing Sentinels Produced a Weak Visible-Test Suite

What happened

The p00639 program processes datasets until it reads `D = 0` :

```
for (; scanf("%lf", &D), D;) {
    /* process one dataset */
}
```

```
}
```

EOF does not safely terminate this loop because the return value of `scanf` is discarded. If input ends after a nonzero `D`, the previous value remains and the program loops indefinitely.

The Tester generated ten candidates across two generations. Several attempted to exercise meaningful geometric behavior but omitted the final `0` sentinel. They correctly received `TIME_LIMIT_EXCEEDED` during deterministic execution. The second generated suite retained only:

```
0.0
```

This is valid input, but it immediately terminates the program and provides no evidence about its main behavior.

How the program handled it in that run

- Every candidate was executed against the accepted C program.
- Candidates that timed out were excluded.
- The Validator reviewed only deterministic survivors and approved the final sentinel-only suite as valid.
- The old policy replaced the entire first suite after one case was considered invalid. Consequently, a useful first-generation survivor was not preserved.
- The resulting suite was shared across prompts, but it offered only weak visible-test evidence. The final Judge was still responsible for evaluation.

Change implemented afterward

Visible-test preparation now separates execution validity from batch quality:

1. Each generated input runs against C before LLM review.
2. Passing slots are preserved; only failed slots are requested again.
3. Runtime failure details remain in deterministic artifacts and are not sent to the Tester.
4. The Validator receives only executable inputs and reviews the batch as a whole for clear source-supported quality problems.
5. A revision names the minimal cases to replace; retained cases stay intact.
6. Replacements run against C before a final batch review.
7. Unresolved slots are reported without discarding other usable cases.

This prevents one defective input from discarding unrelated evidence while allowing the Validator to notice a batch that is valid but too repetitive or trivial. It still cannot infer constraints absent from the C source.

Implications for Future Analysis

- Compare initial and final Judge outcomes to identify repair regressions.
- Treat `SKIPPED` as not evaluable, not as translation success or failure.
- Inspect `SKIPPED` reasons before excluding a problem from the analytical population.
- Report frozen cases, deterministic rejections, Validator-requested replacements, and unresolved replacements separately.
- Do not interpret a valid but trivial visible suite as strong behavioral evidence.
- Validate checker requirements, especially numeric tolerances and special output rules, before freezing Judge eligibility.